# Guía Completa de Ejercicios y Soluciones: Del Simulador al Hardware Cuántico Real y Supercómputo (Módulo 3)

**Narrativa General del Taller:** *"Del simulador al entorno hostil: Domando el hardware cuántico real"*
En este documento se presentan 20 desafíos de complejidad incremental diseñados bajo contextos reales de la industria. Cada problema incluye su formulación práctica, la respuesta analítica detallada para discusión en clase y el código ejecutable en Python utilizando Qiskit.

---

## Bloque 1: Arquitectura Cloud y Primitivas (Fundamentos y Despacho)

### Ejercicio 1: El "Ping" Cuántico (Estado GHZ)

#### 1. Contexto Industrial y Problema

Una startup de telecomunicaciones y ciberseguridad necesita establecer un canal de distribución de claves cuánticas tripartito (QKD) entre tres servidores distribuidos (Nodos A, B y C). Para verificar que el enlace cuántico funciona sin filtraciones intermedias, debes sintetizar un estado máximamente entrelazado de 3 qubits conocido como estado Greenberger-Horne-Zeilinger ($|\mathrm{GHZ}\rangle$) y comprobar mediante simulación local que los tres nodos colapsan siempre al unísono.

#### 2. Respuesta Detallada e Interpretación

El estado $|\mathrm{GHZ}\rangle$ para 3 qubits se define analíticamente como:

$$|\mathrm{GHZ}\rangle = \frac{|000\rangle + |111\rangle}{\sqrt{2}}$$

Para construirlo a partir del estado fundamental $|000\rangle$, aplicamos una cascada de entrelazamiento:

1. Aplicamos una compuerta Hadamard ($H$) en $q_0$, generando la superposición $\frac{|000\rangle+|001\rangle}{\sqrt{2}}$ (recordando la convención *Little-Endian* donde $q_0$ es el bit de más a la derecha).
2. Aplicamos una compuerta $\mathrm{CNOT}_{0\to1}$, que invierte $q_1$ si $q_0 = 1$, obteniendo $\frac{|000\rangle+|011\rangle}{\sqrt{2}}$.
3. Aplicamos una compuerta $\mathrm{CNOT}_{1\to2}$, propagando el entrelazamiento hacia $q_2$ para obtener finalmente $\frac{|000\rangle+|111\rangle}{\sqrt{2}}$.

**Interpretación en el contexto:** La ausencia total de estados intermedios (como $|001\rangle$ o $|101\rangle$) garantiza que la clave generada sea idéntica en las tres ubicaciones simultáneamente. Por la regla de Born, la probabilidad teórica es $P(000) = 0.5$ y $P(111) = 0.5$.

#### 3. Código para Ejecutar

```
from qiskit import QuantumCircuit
from qiskit.primitives import StatevectorSampler

# 1. Construcción del circuito GHZ de 3 qubits
qc_ghz = QuantumCircuit(3, name="GHZ_Ping")
qc_ghz.h(0)         # Superposición en q0
qc_ghz.cx(0, 1)     # Entrelazar q0 -> q1
qc_ghz.cx(1, 2)     # Propagar entrelazamiento q1 -> q2
qc_ghz.measure_all() # Crea automáticamente el registro clásico 'meas'

print("Diagrama del circuito GHZ:")
print(qc_ghz.draw(output='text'))

# 2. Validación en simulador ideal local (1024 disparos)
sampler = StatevectorSampler()
job = sampler.run([qc_ghz], shots=1024)
conteos = job.result()[0].data.meas.get_counts()

print("\nConteos observados:", conteos)
```

## Ejercicio 3: Transición de I+D a Producción (`SamplerV2`)

este

#### 1. Contexto Industrial y Problema

Un banco heredó scripts cuánticos antiguos donde los desarrolladores declaraban manualmente registros clásicos `QuantumCircuit(3, 3)`, medían cable por cable usando `qc.measure(i, i)` y ejecutaban con métodos obsoletos (`backend.run()`). Tu tarea como arquitecto de software cuántico es refactorizar este código al estándar de producción **Primitives V2** (`SamplerV2`), explicando cómo se estructura ahora el retorno de datos.

**2. Respuesta Detallada e Interpretación**

En el paradigma de Qiskit Runtime V2, la interacción con cualquier procesador o simulador se estandariza mediante primitivas:

1. **Declaración simplificada:** No es necesario declarar bits clásicos manualmente si vamos a medir todo el sistema. Al invocar `qc.measure_all()`, Qiskit añade una barrera visual y crea automáticamente un registro clásico de igual tamaño denominado `'meas'`.
2. **Estructura del contenedor de salida (*PUB Result*):** `SamplerV2` permite enviar múltiples circuitos agrupados con distintos parámetros (llamados *Primitive Unified Blocs* o PUBs). Por ello, `job.result()[0]` accede al primer circuito enviado, `.data.meas` accede al registro clásico `'meas'`, y `.get_counts()` devuelve el diccionario de frecuencias acumuladas.

**3. Código para Ejecutar**

```
from qiskit import QuantumCircuit
from qiskit.primitives import StatevectorSampler

# Refactorización moderna: sin declaración manual de bits clásicos
qc_moderno = QuantumCircuit(3)
qc_moderno.h(0)
qc_moderno.cx(0, 1)
qc_moderno.x(2)

# Agrega barrera y crea el registro clásico 'meas' automáticamente
qc_moderno.measure_all()

# Ejecución mediante interfaz estándar Sampler V2
sampler = StatevectorSampler()
pub_result = sampler.run([qc_moderno], shots=1024).result()[0]

# Extracción desde el atributo dinámico 'meas'
conteos = pub_result.data.meas.get_counts()
```

```
print("Registro clásico creado automáticamente:", list(pub_result.data.keys()))
print("Conteos extraídos con estándar V2:", conteos)
```

---

# Bloque 2: Algoritmo de Grover (Búsqueda y Amplificación)

## Ejercicio 5: La Caja Fuerte (Grover de 2 qubits)

este

### 1. Contexto Industrial y Problema

En una auditoría de seguridad informática, debes demostrar cómo un procesador cuántico encuentra una clave PIN de 2 bits ($N = 2^2 = 4$ combinaciones posibles) oculta tras una función de caja negra. El PIN correcto es el estado $|\omega\rangle = |11\rangle$. Mientras un sistema clásico requiere probar hasta 3 veces (y en promedio $2.25$ consultas), debes diseñar el circuito de Grover para hallar la combinación en **exactamente 1 consulta** con un $100\%$ de certeza teórica.

### 2. Respuesta Detallada e Interpretación

Para $N = 4$ y $M = 1$ solución marcada, calculamos el ángulo inicial $\theta$ en el plano 2D generado por $|s'\rangle$ y $|\omega\rangle$:

$$\sin(\theta) = \frac{1}{\sqrt{N}} = \frac{1}{\sqrt{4}} = 0.5 \implies \theta = \arcsin(0.5) = 30^\circ = \frac{\pi}{6} \text{ rad}$$

Al aplicar **1 sola iteración** de Grover ($k = 1$), el teorema de las dos reflexiones nos indica que el vector rota un ángulo neto de $+2\theta$:

$$\theta_1 = (2k + 1)\theta = 3 \times 30^\circ = 90^\circ$$

Como el eje vertical de la solución $|\omega\rangle$ se encuentra exactamente a $90^\circ$, la probabilidad de éxito tras 1 paso es:

$$P_{\text{éxito}}(k = 1) = \sin^2(90^\circ) = 1.0 \quad (100\%)$$

- **Oráculo para $|11\rangle$:** Una simple compuerta $CZ$ (o $H \cdot CX \cdot H$) invierte el signo únicamente cuando ambos qubits valen $1$.
- **Difusor de 2 qubits:** $H^{\otimes 2} \cdot X^{\otimes 2} \cdot CZ \cdot X^{\otimes 2} \cdot H^{\otimes 2}$.

**3. Código para Ejecutar**

```
from qiskit import QuantumCircuit
from qiskit.primitives import StatevectorSampler

qc_pin = QuantumCircuit(2)

# Paso 0: Superposición uniforme inicial |s>
qc_pin.h([0, 1])
qc_pin.barrier()

# Paso 1: Oráculo para |11> (aplica fase -1 solo a |11>)
qc_pin.cz(0, 1)
qc_pin.barrier()

# Paso 2: Operador de Difusión de 2 qubits
qc_pin.h([0, 1])
qc_pin.x([0, 1])
qc_pin.cz(0, 1)
qc_pin.x([0, 1])
qc_pin.h([0, 1])

qc_pin.measure_all()
print(qc_pin.draw(output='text'))

# Simulación local
conteos = StatevectorSampler().run([qc_pin], shots=1024).result()[0].data.meas
print("\nResultado de la auditoría (100% en '11'):", conteos)
```

---

## Ejercicio 6: Enrutamiento Logístico (qiskit oracle de 3 qubits)

este

**1. Contexto Industrial y Problema**

Una empresa de transporte urbano evalúa 8 rutas posibles codificadas en 3 bits ($N = 8$). Debido a un corte de tránsito imprevisto, solo la ruta número 5 (en binario `'101'`) cumple con todas las restricciones de entrega a tiempo. Construye una función modular en Qiskit que implemente el **Oráculo de Fase** capaz de reconocer exclusivamente el estado $|101\rangle$ e invertir el signo de su amplitud sin alterar ningún otro estado.

**2. Respuesta Detallada e Interpretación**

El oráculo de fase debe implementar el operador unitario $U_\omega = I - 2|101\rangle\langle 101|$.
En Qiskit disponemos de la compuerta NOT multi-controlada (`mcx`), la cual se activa únicamente cuando todos los qubits de control están en el estado $|1\rangle$. Para convertir esta inversión de bit en una inversión de fase pura (compuerta $Z$ multi-controlada), conjugamos el qubit objetivo con compuertas Hadamard ($H \cdot X \cdot H = Z$).

Siguiendo la convención *Little-Endian* ($|q_2 q_1 q_0\rangle = |101\rangle$):

- $q_0 = 1$ (no requiere inversión previa).
- $q_1 = 0$ (**requiere envolvente con compuerta** $X$ antes y después del bloque central para activar el control cuando su estado original es $0$).
- $q_2 = 1$ (actúa como objetivo rodeado de compuertas $H$).

**3. Código para Ejecutar**

```
from qiskit import QuantumCircuit
from qiskit.quantum_info import Statevector
import numpy as np

def construir_oraculo_ruta(estado_objetivo: str) -> QuantumCircuit:
    n_qubits = len(estado_objetivo)
    qc = QuantumCircuit(n_qubits, name=f"Oráculo |{estado_objetivo}>")

    # 1. Pre-procesamiento: X en los qubits que deben valer '0' (Little-Endiar
    for qubit, bit in enumerate(reversed(estado_objetivo)):
        if bit == '0':
            qc.x(qubit)

    # 2. Disparo de fase: H + mcx + H sobre el último qubit
    ultimo = n_qubits - 1
    qc.h(ultimo)
    qc.mcx(list(range(ultimo)), ultimo)
```

```
    qc.h(ultimo)

    # 3. Restauración: X en los mismos qubits para devolverlos a su estado ori
    for qubit, bit in enumerate(reversed(estado_objetivo)):
        if bit == '0':
            qc.x(qubit)

    return qc

oraculo_101 = construir_oraculo_ruta("101")
print(oraculo_101.draw(output='text'))

# Verificación analítica sobre una superposición uniforme
qc_test = QuantumCircuit(3)
qc_test.h([0, 1, 2])
qc_test.compose(oraculo_101, inplace=True)

 vector_estado = Statevector.from_instruction(qc_test)
for idx, amp in enumerate(vector_estado.data):
    binario = format(idx, '03b')
    print(f"Estado |{binario}> : Amplitud = {np.real(amp):+.4f}")
```

---

## Ejercicio 9: La Trampa de la Sobre-rotación

este

**1. Contexto Industrial y Problema**

Un ingeniero cuántico junior pensó que el algoritmo de Grover funcionaba como los algoritmos clásicos iterativos (donde iterar más veces siempre mejora o mantiene la convergencia) y configuró $k = 4$ iteraciones para buscar 1 elemento en una base de 8 registros ($N = 8$). Para su sorpresa, el algoritmo arrojó resultados peores que elegir al azar. Demuestra mediante un barrido paramétrico desde $k = 0$ hasta $k = 6$ por qué ocurrió este colapso.

**2. Respuesta Detallada e Interpretación**

El algoritmo de Grover es una **rotación unitaria rígida** en el plano 2D. La probabilidad exacta de éxito tras $k$ pasos sigue la ley sinusoidal:

$$P_{\text{éxito}}(k) = \sin^2\left((2k+1)\theta\right)$$

donde $\theta = \arcsin(1/\sqrt{8}) \approx 20.70°$. Veamos la evolución angular:

- $k = 0$: $\theta_0 = 20.70° \implies P = 12.5\%$
- $k = 1$: $\theta_1 = 62.11° \implies P = 78.1\%$
- $k = 2$ (**Óptimo**): $\theta_2 = 103.52° \implies P = 94.5\%$
- $k = 3$ (**Sobre-rotación**): $\theta_3 = 144.92° \implies P = 33.0\%$
- $k = 4$ (**Colapso casi total**): $\theta_4 = 186.33° \implies P = \sin^2(186.33°) \approx 1.2\%$

En $k = 4$, el vector de estado ha girado un poco más de media vuelta ($180°$), quedando casi perfectamente alineado con el eje horizontal de las respuestas incorrectas $|s'\rangle$. En computación cuántica es tan crítico saber cómo construir el circuito como saber **cuándo detenerlo**.

### 3. Código para Ejecutar

```
from qiskit import QuantumCircuit
from qiskit.quantum_info import Statevector
import numpy as np

n = 3
theta = np.arcsin(1 / np.sqrt(2**n))

print(f"{'Iteraciones (k)':<18}{'Ángulo (grados)':<18}{'Prob. Éxito |110> (%)'
print("=" * 58)

for k in range(7):
    qc = QuantumCircuit(n)
    qc.h(range(n))
    for _ in range(k):
        qc.compose(construir_oraculo_ruta("110"), inplace=True)
        qc.compose(construir_difusor(n), inplace=True)

    prob_110 = Statevector.from_instruction(qc).probabilities_dict().get("110"
    angulo_deg = np.degrees((2 * k + 1) * theta)
    barra = "█" * int(prob_110 // 4)
    print(f"{k:<18}{angulo_deg:<18.2f}{prob_110:6.2f}%  {barra}")
```

## Ejercicio 10: Escalando la Base de Datos (Grover de 5 qubits)

este

### 1. Contexto Industrial y Problema

Una aseguradora necesita localizar una póliza específica dentro de un bloque desestructurado de $N = 32$ registros ($n = 5$ qubits). La póliza buscada corresponde a la cadena binaria `'10110'`. Calcula el número óptimo de iteraciones $k_{\text{óptimo}}$, ensambla el circuito de 5 qubits de forma modular y verifica que la probabilidad de éxito supera el $99\%$.

### 2. Respuesta Detallada e Interpretación

Para $n = 5$ qubits ($N = 2^5 = 32$), el ángulo elemental disminuye a:

$$\theta = \arcsin\left(\frac{1}{\sqrt{32}}\right) = \arcsin(0.17677) \approx 0.1777 \text{ rad} \approx 10.18^\circ$$

Calculamos el número óptimo de iteraciones:

$$k_{\text{óptimo}} = \text{round}\left(\frac{\pi}{4}\sqrt{32}\right) = \text{round}(4.4428) = 4 \text{ iteraciones}$$

Con $k = 4$ iteraciones, el ángulo acumulado es $(2(4) + 1) \times 10.18^\circ = 9 \times 10.18^\circ = 91.64^\circ$, lo que nos deja a tan solo $1.64^\circ$ de la vertical perfecta ($90^\circ$). La probabilidad teórica de hallar la póliza en solo 4 consultas (frente a las 16 consultas promedio clásicas) alcanza:

$$P_{\text{éxito}}(k = 4) = \sin^2(91.64^\circ) \approx 99.92\%$$

### 3. Código para Ejecutar

```
import numpy as np
from qiskit import QuantumCircuit
from qiskit.primitives import StatevectorSampler

n_qubits = 5
N = 2**n_qubits
```

```
objetivo = "10110"
k_opt = int(np.round(np.pi / 4 * np.sqrt(N)))

qc_5q = QuantumCircuit(n_qubits)
qc_5q.h(range(n_qubits))

oraculo_5q = construir_oraculo_ruta(objetivo)
difusor_5q = construir_difusor(n_qubits)

for _ in range(k_opt):
    qc_5q.compose(oraculo_5q, inplace=True)
    qc_5q.compose(difusor_5q, inplace=True)

qc_5q.measure_all()

shots = 4096
conteos = StatevectorSampler().run([qc_5q], shots=shots).result()[0].data.meas
prob_exito = conteos.get(objetivo, 0) / shots * 100

print(f"Espacio de búsqueda: N = {N} registros ({n_qubits} qubits)")
print(f"Iteraciones óptimas aplicadas: k = {k_opt}")
print(f"Probabilidad empírica de éxito para |{objetivo}>: {prob_exito:.2f}% ({
```

---

# Bloque 3: Topología, Ruido y Supercómputo (HPC & GPUs)

## Ejercicio 11: Compilando para Hardware Restringido (`CouplingMap`)

util para visualizar

#### 1. Contexto Industrial y Problema

Un grupo de ingeniería desarrolla un circuito que entrelaza directamente el qubit $q_0$ con el qubit $q_2$. Sin embargo, al enviarlo al procesador superconductor real, descubren que los qubits físicos están dispuestos en una línea recta donde $q_0$ solo está conectado con $q_1$, y $q_1$ solo está conectado con $q_2$ ($0 \leftrightarrow 1 \leftrightarrow 2$). Demuestra cómo el transpilador de Qiskit resuelve esta limitación geométrica e identifica el costo en compuertas de 2 qubits.

#### 2. Respuesta Detallada e Interpretación

En los procesadores superconductores de arquitectura *Heavy-Hex* de IBM, cada qubit tiene solo 2 o 3 vecinos físicos directos para evitar interferencia electromagnética cruzada (*crosstalk*).
Cuando solicitamos una compuerta $\mathrm{CNOT}_{0\to 2}$ en una topología lineal $0 \leftrightarrow 1 \leftrightarrow 2$, el compilador debe insertar una compuerta **SWAP** entre $q_0$ y $q_1$ (o entre $q_1$ y $q_2$) para acercar los estados lógicos a posiciones físicamente adyacentes.

Dado que la compuerta SWAP no es una operación física nativa de microondas, se descompone exactamente en **3 compuertas CNOT alternadas**:

$$\mathrm{SWAP}_{0,1} = \mathrm{CNOT}_{0\to 1} \cdot \mathrm{CNOT}_{1\to 0} \cdot \mathrm{CNOT}_{0\to 1}$$

Así, una sola CNOT distante se convierte en hasta 4 compuertas CNOT físicas en el hardware real.

**3. Código para Ejecutar**

```
from qiskit import QuantumCircuit, transpile
from qiskit.transpiler import CouplingMap
"""
qiskit draw para dibujar circuito y dibujar circuito
"""
# Circuito ideal que intenta conectar q0 directamente con q2
qc_distante = QuantumCircuit(3)
qc_distante.h(0)
qc_distante.cx(0, 2) # Conexión no adyacente

# Definimos topología lineal 0 <-> 1 <-> 2
mapa_lineal = CouplingMap([[0, 1], [1, 0], [1, 2], [2, 1]])

# Transpilamos con optimization_level=0 para ver la inserción pura del SWAP
qc_mapeado = transpile(qc_distante, coupling_map=mapa_lineal, basis_gates=['h'

print("Circuito ideal (1 CNOT):")
print(qc_distante.draw(output='text'))
print("\nCircuito adaptado a topología lineal (incluye descomposición de SWAP
print(qc_mapeado.draw(output='text'))
print("Total de compuertas CNOT tras enrutamiento:", qc_mapeado.count_ops().ge
```

## Ejercicio 12: El Costo Térmico del Enrutamiento (Optimización Nivel 0 vs Nivel 3)

Dejar de bonus

### 1. Contexto Industrial y Problema

Cada compuerta de 2 qubits ($\mathrm{CNOT}$) en un procesador real introduce aproximadamente un $1\%$ a $2\%$ de error físico. Antes de lanzar el algoritmo de Grover de 3 qubits a producción sobre una topología lineal, debes realizar un *benchmark* del compilador comparando el nivel de optimización nulo (`optimization_level=0`) frente al nivel de síntesis algebraica agresiva (`optimization_level=3`), cuantificando cuántas compuertas CNOT logras ahorrar.

### 2. Respuesta Detallada e Interpretación

El circuito de Grover de 3 qubits ($k = 2$ iteraciones) contiene 4 compuertas Toffoli (`mcx` de 2 controles: dos en los oráculos y dos en los difusores). Cada compuerta Toffoli requiere al menos 6 compuertas CNOT en conectividad total, y bastantes más en conectividad lineal debido a los SWAPs.

- En **`optimization_level=0`**, el transpilador simplemente traduce cada bloque de forma literal e inserta SWAPs heurísticos sin cancelar compuertas adyacentes redundantes, disparando el conteo de CNOTs por encima de 50.
- En **`optimization_level=3`**, Qiskit aplica síntesis unitaria de bloques de 2 qubits (descomposición KAK), reordena compuertas que conmutan y fusiona rotaciones de 1 qubit, logrando reducir sustancialmente la cantidad de CNOTs y protegiendo la coherencia del estado cuántico.

### 3. Código para Ejecutar

```
from qiskit import QuantumCircuit, transpile
from qiskit.transpiler import CouplingMap

# Ensamblamos Grover completo de 3 qubits (k=2)
qc_grover = QuantumCircuit(3)
qc_grover.h([0, 1, 2])
for _ in range(2):
    qc_grover.compose(construir_oraculo_ruta("110"), inplace=True)
```

```
    qc_grover.compose(construir_difusor(3), inplace=True)
qc_grover.measure_all()

mapa_lineal = CouplingMap([[0, 1], [1, 0], [1, 2], [2, 1]])
compuertas_nativas = ['cx', 'id', 'rz', 'sx', 'x']

# Comparativa de compilación
for nivel in [0, 1, 2, 3]:
    t_qc = transpile(qc_grover, coupling_map=mapa_lineal, basis_gates=compuert
    cnots = t_qc.count_ops().get('cx', 0)
    print(f"Nivel de Optimización {nivel} -> Profundidad: {t_qc.depth():3d} |
```

---

## Ejercicio 13: Prueba de Estrés Ambiental (Simulación con Ruido Local)

absolutamente si

**1. Contexto Industrial y Problema**

Antes de gastar presupuesto en el procesador real, el líder técnico te pide simular cómo se comportará el circuito de Grover transpilado (obtenido en el Ejercicio 12 con `optimization_level=3`) si el hardware físico presenta una tasa de error de despolarización del $2.0\%$ en cada compuerta CNOT. Construye el modelo de ruido en `qiskit_aer`, ejecuta la simulación ruidosa y compara el porcentaje de éxito con la predicción teórica de supervivencia de compuertas.

**2. Respuesta Detallada e Interpretación**

El canal de error de despolarización (*depolarizing error*) modela el proceso mediante el cual un par de qubits pierde su coherencia con probabilidad $p$ y es reemplazado por el estado completamente mixto (ruido blanco uniforme $I/4$).

Si el circuito transpilado en nivel 3 posee $N_{\text{CNOT}} = 39$ compuertas de 2 qubits y cada una tiene una fidelidad de $(1 - 0.02) = 0.98$, la probabilidad aproximada de que el circuito completo se ejecute sin sufrir ningún fallo despolarizante es:

$$P_{\text{sin fallo}} \approx (0.98)^{39} \approx 45.48\%$$

Sumando la probabilidad residual de caer en $|110\rangle$ por puro azar cuando ocurre un error ($\frac{1}{8}$ del $54.52\%$ restante $\approx 6.8\%$), esperamos que el pico de Grover caiga desde el $94.5\%$ ideal hasta aproximadamente el $50\% - 52\%$.

**3. Código para Ejecutar**

```
from qiskit_aer import AerSimulator
from qiskit_aer.noise import NoiseModel, depolarizing_error

# Transpilamos en nivel 3 (39 CNOTs)
t_opt3 = transpile(qc_grover, coupling_map=mapa_lineal, basis_gates=compuertas

# Creamos el modelo de ruido con 2% de error en compuertas de 2 qubits ('cx')
modelo_ruido = NoiseModel()
error_cnot = depolarizing_error(0.020, 2)
modelo_ruido.add_all_qubit_quantum_error(error_cnot, ['cx'])

# Ejecutamos en AerSimulator con ruido
sim_ruidoso = AerSimulator(noise_model=modelo_ruido)
conteos_ruido = sim_ruidoso.run(t_opt3, shots=4096, seed_simulator=42).result(

prob_110_ruido = conteos_ruido.get('110', 0) / 4096 * 100
print("Conteos bajo estrés ambiental (2% error CNOT):", conteos_ruido)
print(f"\nProbabilidad ideal de |110>:   94.53%")
print(f"Probabilidad con ruido físico: {prob_110_ruido:.2f}% (El estado |110>
```

---

## Ejercicio 14: El Muro de Memoria RAM (Escalamiento Clásico)

dejar este está bonito

**1. Contexto Industrial y Problema**

Un laboratorio farmacéutico desea simular moléculas grandes en sus servidores clásicos antes de usar computadores cuánticos. Te piden escribir un script que calcule exactamente cuánta memoria RAM consume el vector de estado desde $n = 10$ hasta $n = 40$ qubits, y que mida en vivo el tiempo que tarda un único núcleo de CPU en simular 1 iteración de Grover entre $n = 14$ y $n = 22$ qubits, demostrando por qué el tiempo y la memoria crecen como $\mathcal{O}(2^n)$.

**2. Respuesta Detallada e Interpretación**

Para simular exactamente un circuito cuántico de $n$ qubits mediante el método de vector de estado (*statevector*), un computador clásico debe almacenar en memoria RAM las $2^n$ amplitudes complejas ($\alpha_0, \alpha_1, \ldots, \alpha_{2^n-1}$).
En precisión doble estándar (`complex128`), cada número complejo ocupa $16\ \text{bytes}$ ( parte real + parte imaginaria):

Cada vez que agregamos **un solo qubit**, la memoria requerida y el número de multiplicaciones matriciales **se duplican**:

- (Instantáneo en cualquier laptop).
- (Colapsa una laptop estándar).
- (Requiere un supercomputador distribuido).

**3. Código para Ejecutar**

```
import time
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator

print("=== 1. PROYECCIÓN DE MEMORIA RAM DEL VECTOR DE ESTADO ===")
for n_mem in [10, 20, 26, 30, 34, 40]:
    bytes_req = (2**n_mem) * 16
    if bytes_req < 1024**2:
        mem_str = f"{bytes_req / 1024:.1f} KB"
    elif bytes_req < 1024**3:
        mem_str = f"{bytes_req / (1024**2):.1f} MB"
    elif bytes_req < 1024**4:
        mem_str = f"{bytes_req / (1024**3):.1f} GB"
    else:
        mem_str = f"{bytes_req / (1024**4):.1f} TB"
    print(f"  n = {n_mem:2d} qubits | Dimensión = {2**n_mem:15,d} | Memoria RA

print("\n=== 2. BENCHMARK EN VIVO EN CPU MONOHILO (1 NÚCLEO) ===")
sim_cpu1 = AerSimulator(method="statevector", device="CPU", max_parallel_threa

for n in [14, 16, 18, 20, 22]:
    qc = QuantumCircuit(n)
    qc.h(range(n))
```

```
    qc.mcx(list(range(n - 1)), n - 1)
    qc.measure_all()

    t0 = time.time()
    sim_cpu1.run(qc, shots=200).result()
    dt = time.time() - t0
    print(f"  n = {n:2d} qubits ({2**n:9,d} amplitudes) -> Tiempo CPU (1 hilo)
```

---

## Ejercicio 15: Aceleración con cuQuantum (GPU vs CPU)

revisar si tenemos el hardware

### 1. Contexto Industrial y Problema

Para superar el cuello de botella del Ejercicio 14 y simular sistemas de 26 a 32 qubits en segundos, el centro de supercómputo ha instalado tarjetas gráficas NVIDIA A100 con el SDK `cuQuantum` (`qiskit-aer-gpu`). Debes escribir el script de evaluación que detecte si hay una GPU activa, compare el tiempo de simulación frente a la CPU y explique físicamente por qué las GPUs logran aceleraciones de más de en simulación cuántica.

### 2. Respuesta Detallada e Interpretación

Aplicar una compuerta cuántica sobre un vector de números complejos exige actualizar millones de pares de amplitudes independientes en memoria. Este problema no está limitado por la velocidad del reloj del procesador, sino por el **ancho de banda de la memoria (*memory bandwidth*)**:

- Una CPU convencional transfiere datos desde la RAM a .
- Una GPU de supercómputo (como la NVIDIA A100) cuenta con memoria HBM2e que transfiere datos a más de () y miles de núcleos CUDA que actualizan los pares de amplitudes simultáneamente.
- *Nota de ingeniería:* Para pocos qubits (), la CPU suele ser igual o más rápida porque inicializar el contexto CUDA y enviar datos por el bus PCIe tiene un costo fijo inicial (*overhead*). Pero a partir de , la GPU domina exponencialmente.

### 3. Código para Ejecutar

```
import time
import numpy as np
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator

sim_cpu = AerSimulator(method="statevector", device="CPU", max_parallel_threac

# Verificamos disponibilidad de GPU CUDA en el entorno actual
try:
    sim_gpu = AerSimulator(method="statevector", device="GPU")
    sim_gpu.run(QuantumCircuit(2)).result()
    gpu_activa = True
    print("GPU NVIDIA detectada exitosamente. Ejecutando benchmark en vivo..."
except Exception:
    gpu_activa = False
    print("GPU no detectada en esta máquina. Se mostrarán tiempos reales de CP

qubits_test = [16, 18, 20, 22, 24]
tiempos_a100_ref = {16: 0.015, 18: 0.003, 20: 0.004, 22: 0.006, 24: 0.014}

print(f"\n{'Qubits':<8}{'Amplitudes':<14}{'CPU (1 núcleo)':<18}{'GPU A100':<16
print("=" * 75)

for n in qubits_test:
    qc = QuantumCircuit(n)
    qc.h(range(n))
    qc.mcx(list(range(n - 1)), n - 1)
    qc.measure_all()

    t0 = time.time()
    sim_cpu.run(qc, shots=200).result()
    t_cpu = time.time() - t0

    if gpu_activa:
        t0 = time.time()
        sim_gpu.run(qc, shots=200).result()
        t_gpu = time.time() - t0
    else:
        t_gpu = tiempos_a100_ref[n]
```

```
    speedup = t_cpu / t_gpu
    print(f"{n:<8}{2**n:<14,d}{t_cpu:<18.4f}s{t_gpu:<16.4f}s{speedup:6.1f}x")
```

---